

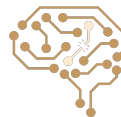
A magnifying glass is held in the foreground, its lens focusing on a sunset scene. The background is a blurred landscape with a green field and a bright orange and yellow sky. The magnifying glass's frame is dark and circular, and its handle is visible at the bottom left.

Local Search Part 2

Arash Marioriyad

1 Ordibehesht 1405

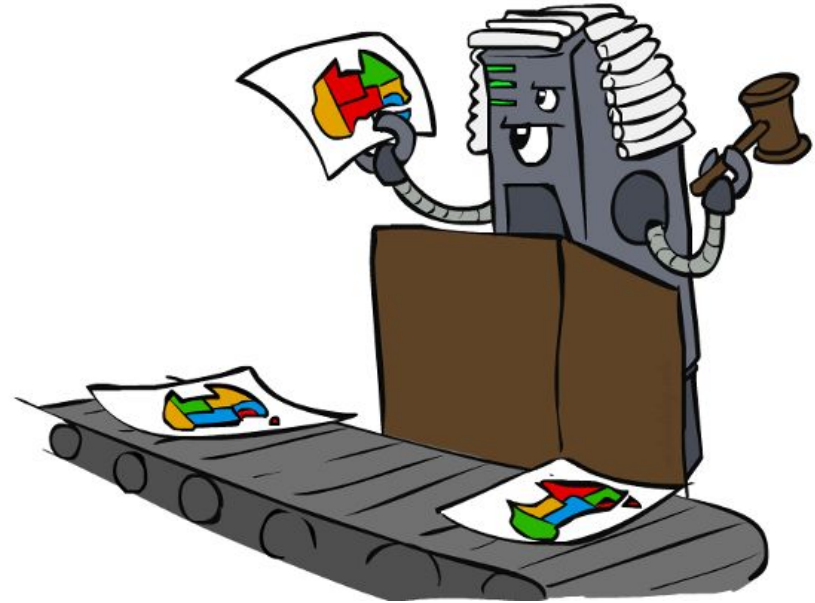
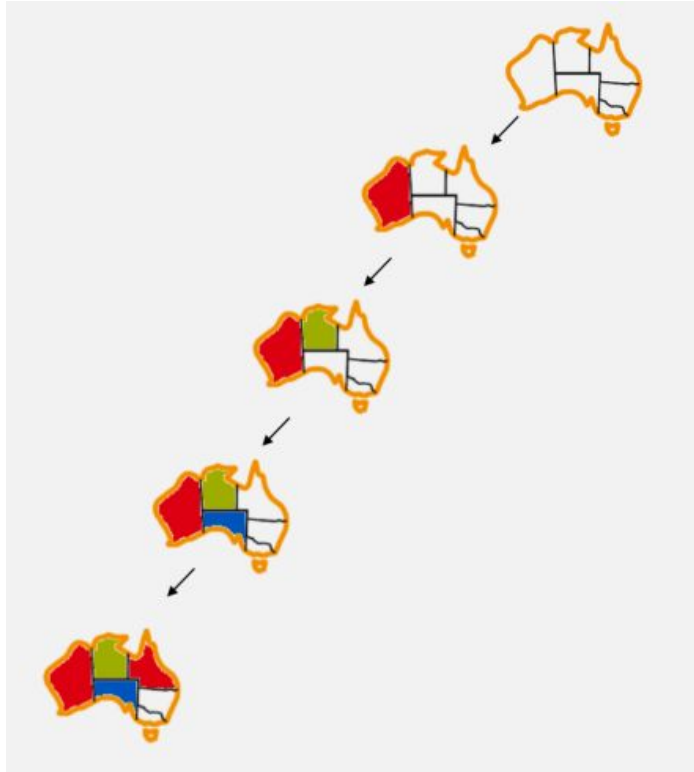
CE 417-Artificial
Intelligence
Sharif Univ. of Tech.



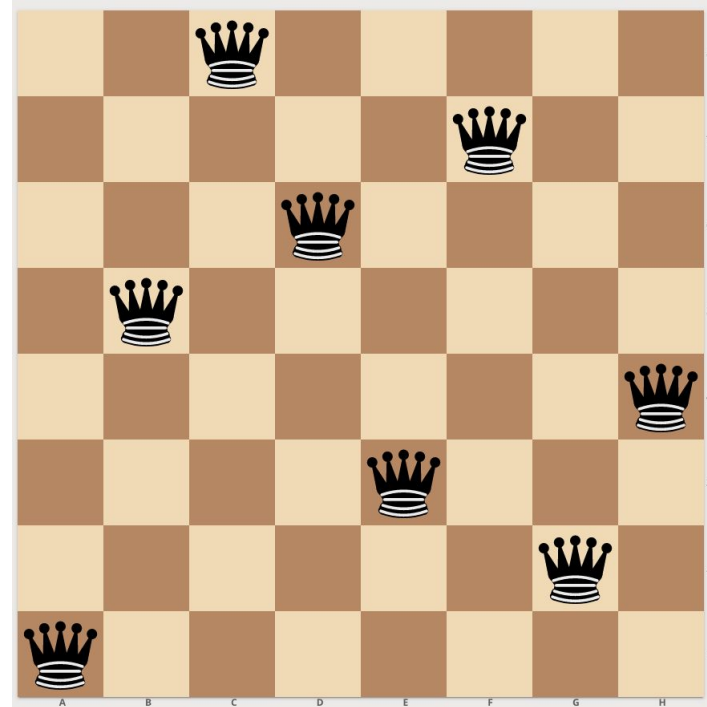
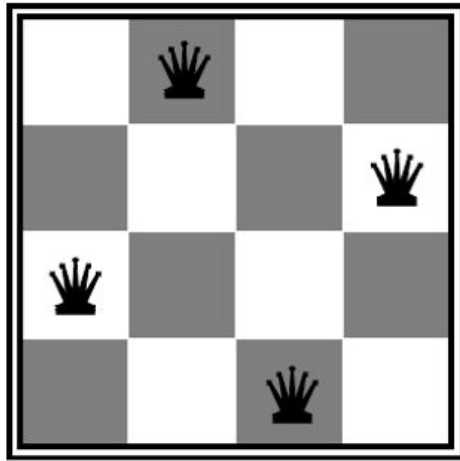
Partial State

VS

Complete State



Example: N-Queens Problem



The Twist

- Goal states are complete states.
- All complete states exist at the lowest level of the search tree.
- Every path to a goal state is considered optimal (path optimality is irrelevant).
- Previous concepts (A^* , heuristics, etc.) no longer apply.
- Question: Why not just search exclusively among complete states?
- Result: Tree searching is no longer necessary.



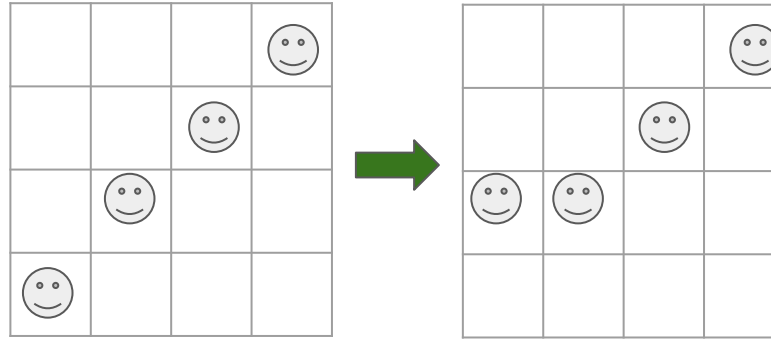
Local Search

- State Space: All complete configurations (complete states)
- State State: A random complete state
- The local search algorithm is just an iterative improvement: Keep a single "current" state, try to improve it by going to a neighbor state
- Local search improves a single option at each time (no frontier)
- Memory-less: only remember current state



Neighboring in Local Search

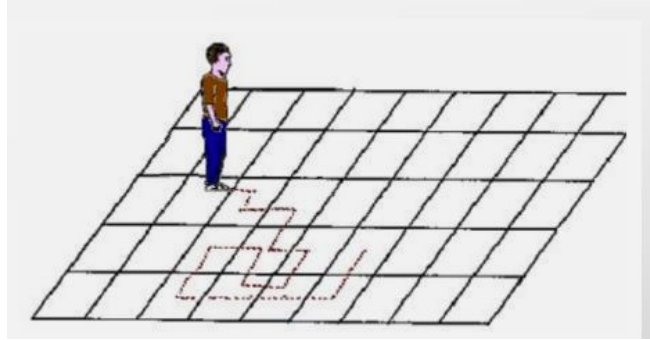
- Neighbors of a state: States that are different in only one variable



- Question: Which neighbor must be choose?



Random Walk

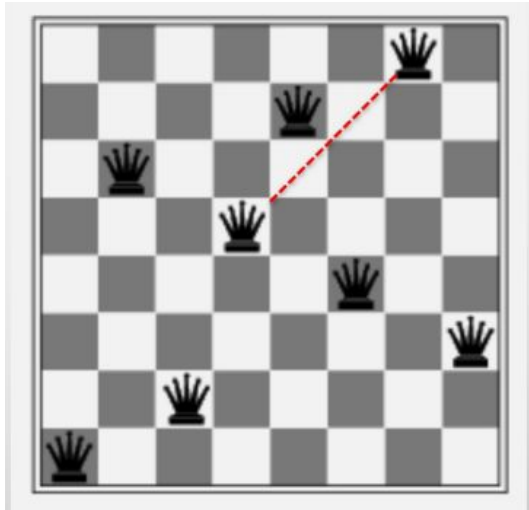


- Randomly pick a neighbor of the current state
- Random Walk algorithm is asymptotically complete (as the number of iterations increases, the probability that the algorithm has found the goal state approaches 1)



Fitness Function

- How good is each state?
- For N-Queens: # pairs of incompatible queens
- What about map coloring problem?



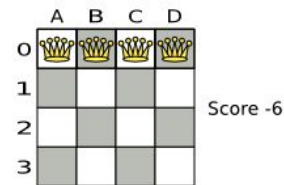
Hill-climbing

- Start from a random complete state
- Repeat: move to the best neighbor that is better than current state
- If no neighbors better than current state, quit
- Also called steepest-ascent hill climbing
- Complete?

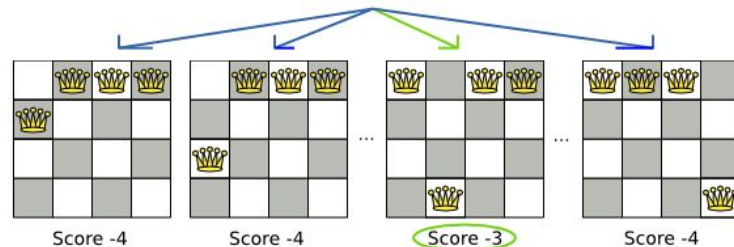


Hill-climbing on N-Queens

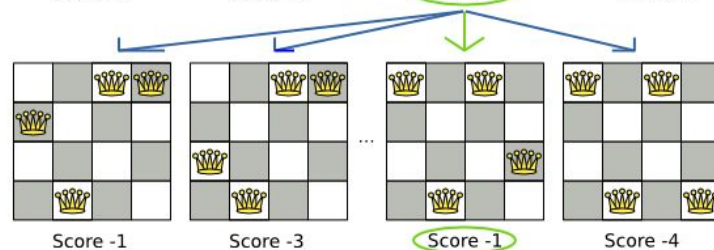
Step 0



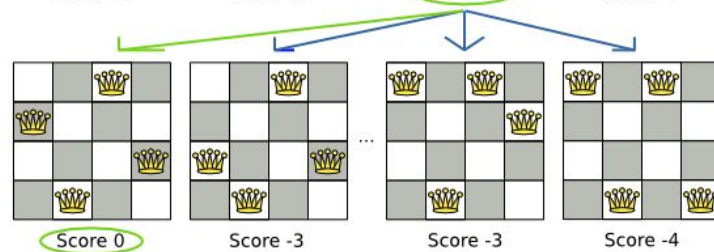
Step 1



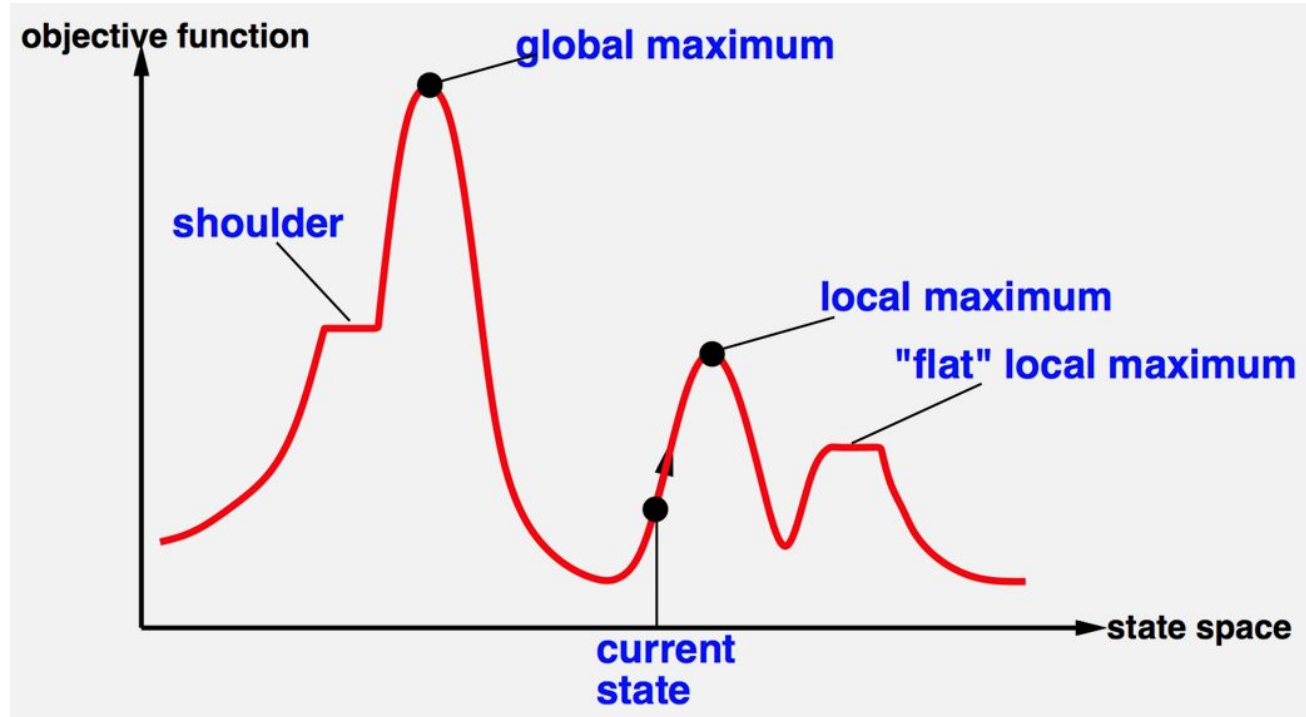
Step 2



Step 3



Hill Climbing Hells



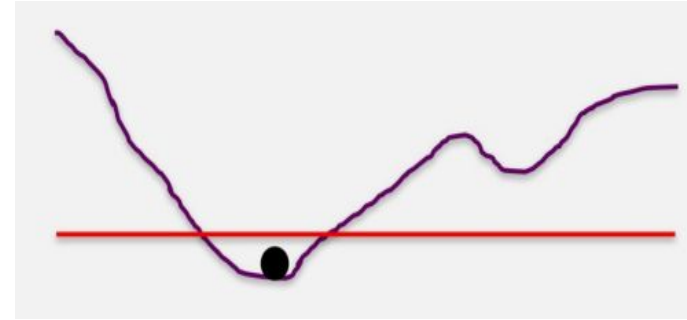
Hill-climbing Variations

- + Limited Number of Consecutive Sideway Moves
- + Tabu List (a fixed length queue)
- + Random Walk
- + Random Restarts



Simulated Annealing (SA)

- Idea: escape local maxima by allowing some “bad” moves
- but gradually decrease their size and frequency
- Imagine letting a ball roll downhill on the function surface
- Now shake the surface, while the ball rolls
- Gradually reducing the amount of shaking



Simulated Annealing (SA) Algorithm

function Simulated-Annealing(*problem*, *schedule*) **returns**
solution state

current $\leftarrow$ Make-Node(Initial-State[*problem*])

for *t* $\leftarrow$ 1 **to** *infinity*

T $\leftarrow$ *schedule*[*t*]

if *T* = 0 **then** **return** *current*

next $\leftarrow$ Random-Successor(*current*)

$\Delta E \leftarrow$ f-Value[*next*] - f-Value[*current*]

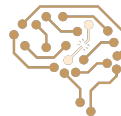
if $\Delta E > 0$ **then** *current* $\leftarrow$ *next*

else *current* $\leftarrow$ *next* with probability $e^{\Delta E/T}$

end

- ▶ Pick a random successor of the current state
- ▶ If it is better than the current state go to it
- ▶ Otherwise, accept the transition with a probability

$T(t) = \text{schedule}[t]$ is a decreasing series

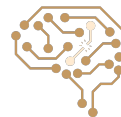
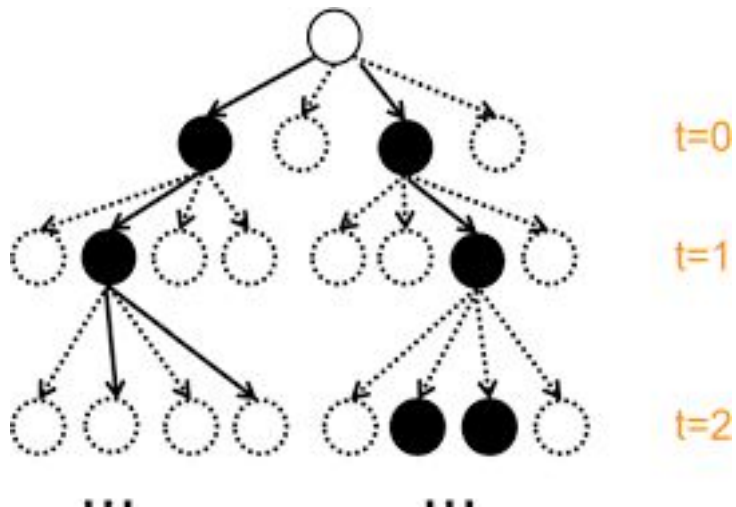


Local Beam Search

- Instead of just one in hill-climbing and simulated annealing
- Keep track of k states simultaneously
 - Initially: k randomly selected states
 - Next: determine all neighbors of all k states
 - If any of successors is goal: finish
 - Else select k best from the all neighbors and repeat
- Not same as k independent random-start searches run in parallel!



Local Beam Search Example



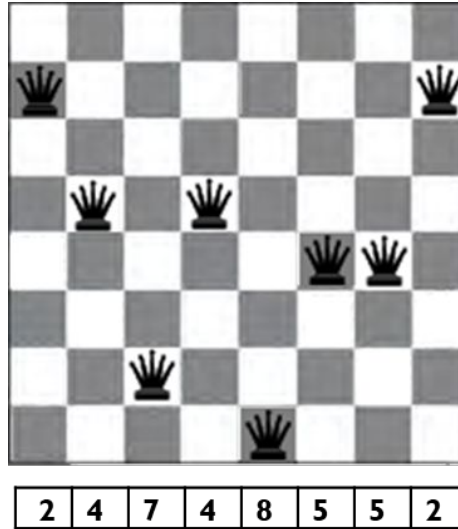
Genetic algorithms

- No neighboring concept
- Start with k randomly generated states (initial population)
- A state is represented as a string over a finite alphabet (Chromosome), often a string of 0s and 1s
- Evaluation function (fitness function)
- Produce the next generation of states by **selection**, **crossover**, and **mutation**



Chromosome & Fitness: 8-queens Problem

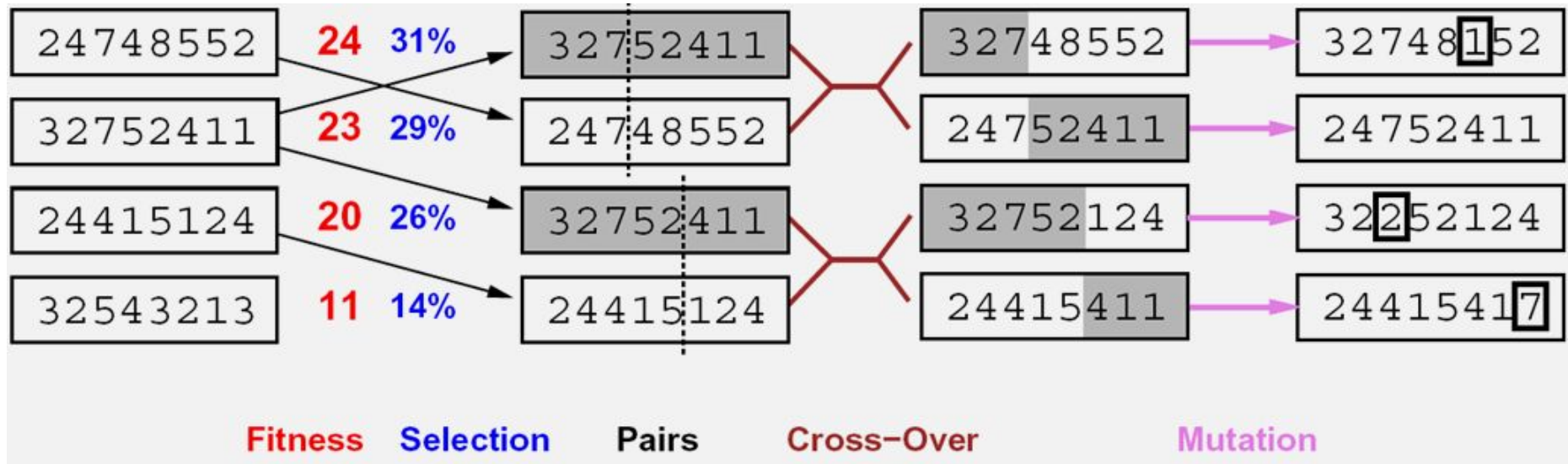
- Describe the individual (or state) as a string



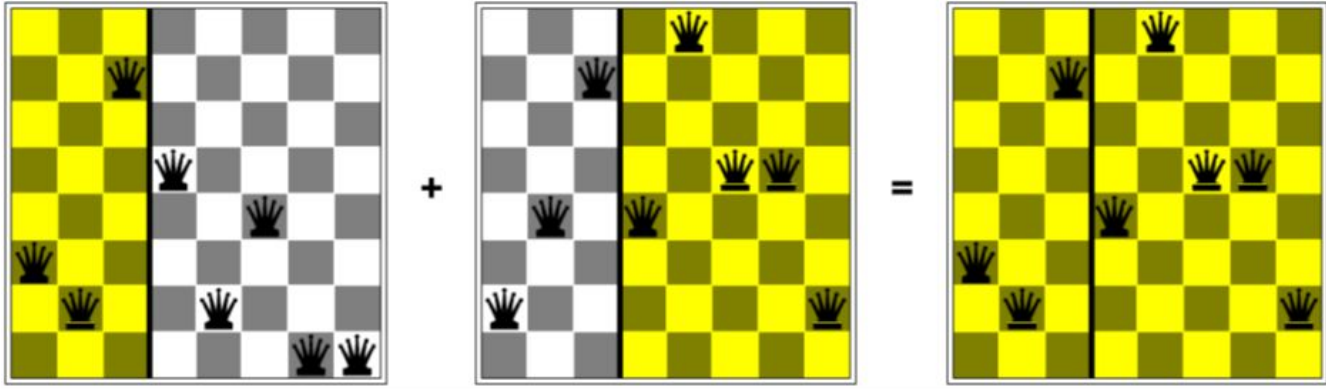
- Fitness function: number of non-attacking pairs of queens (24 for above figure)



Genetic algorithms



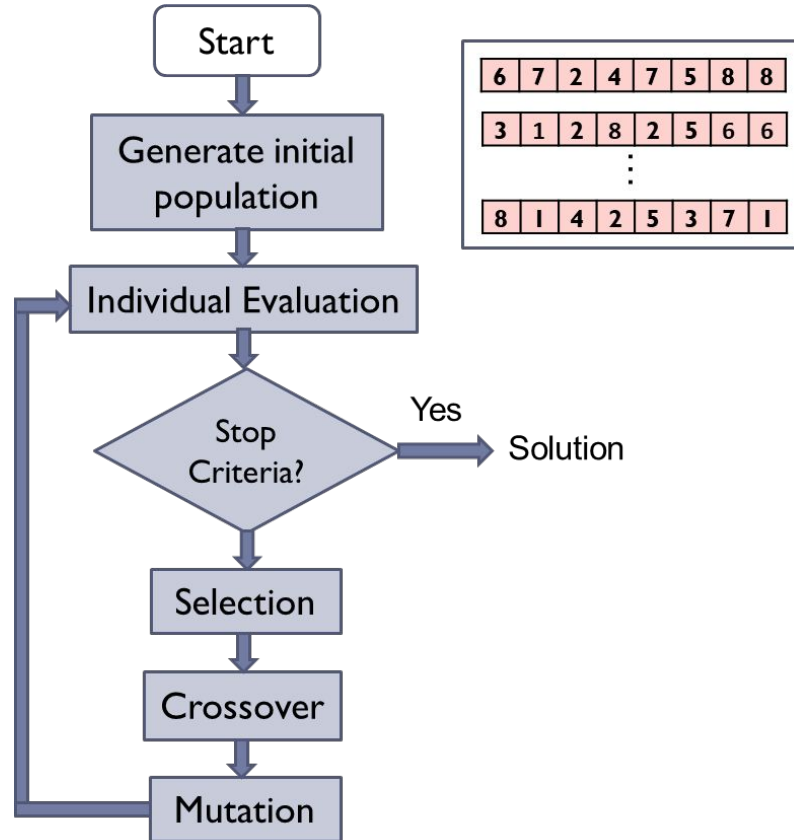
Example: N-Queens



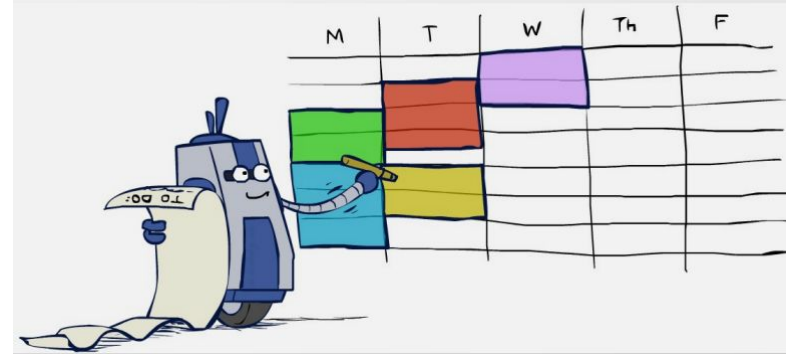
- Does crossover make sense here?
- What would mutation be?
- What would a good fitness function be?



Genetic Algorithm Diagram



Next Session: Constraint Satisfaction Problem (CSP)



$$\begin{array}{r} \text{TWO} \\ + \text{TWO} \\ \hline \text{FOUR} \end{array}$$

